NEON HUB

Full-Stack Media Tracking Platform

Software Architecture & Requirements Document

Stack: React · Vite · Node.js · Express · PostgreSQL · Bootstrap 5
APIs: TMDB · RAWG
Deployment: Netlify · Render · Neon DB · Cloudinary

Version 1.0 · June 2026

Table of Contents

1. Project Overview & Vision

1.1 What Is Neon Hub?

Neon Hub is a full-stack, production-grade media tracking platform where users can discover, save, track, rate, and review Movies, TV Series, and Games from a single unified interface. It combines external API integrations (TMDB, RAWG), relational database persistence, JWT-based authentication, and a rich analytics dashboard — all packaged in a modern, dark-themed UI.

This is NOT a tutorial clone. Neon Hub is designed for real users, real traffic, and long-term evolution as a flagship portfolio project that demonstrates senior-level full-stack engineering.

1.2 Core Value Proposition

- One platform to track movies, TV series, and games — no switching between apps
- Full progress tracking: episodes watched, hours played, completion %
- Personal analytics dashboard with genre insights and activity history
- Custom curated lists and public review system
- Clean, fast, mobile-responsive interface with a distinctive neon aesthetic

1.3 Target Users

User Type	Primary Use Case	Key Features Used
Casual Consumer	Track what they've watched/played	Library, Status, Favorites
Power Tracker	Detailed logs with ratings & reviews	Dashboard, Reviews, Stats
List Builder	Curate and share collections	Custom Lists, Discover
Discovery Seeker	Find new media via trending/genre	Discover, Search, Trending

1.4 Project Constraints

- Free-tier deployment only (Netlify + Render + Neon DB + Cloudinary)
- No paid third-party services or premium APIs
- Solo developer or small team with AI agent assistance
- Must be production-ready, not just a demo

2. Gap Analysis — What Was Missing

The original project breakdown is strong and well-structured. The analysis below identifies gaps — missing features, under-specified systems, and architectural risks — that must be addressed before development begins.

2.1 Missing Features Identified

Gap Area	What Was Missing	Severity	Resolution
Notifications	No notification system for new episodes, game releases	Medium	Add notification_prefs table + in-app alert banner
Search UX	No debounce, no search history, no "no results" state	High	Implement debounced search hook + empty state UI
Pagination/Infinite Scroll	API responses not paginated — will break at scale	High	Add cursor/page param to all list endpoints
Image Optimization	Avatar uploads lack CDN or persistent storage — files lost on Render redeploy	Medium	Use Cloudinary free tier for upload, auto-resize (via transformation URL params), and CDN delivery
Refresh Tokens	Only access JWT, no refresh flow — forces re-login	High	Add refresh_tokens table + /auth/refresh endpoint
Email Verification	No email confirm on register — allows fake accounts	Medium	Add email_verified flag + verification flow (Phase 2+)
Password Reset	No "forgot password" flow defined anywhere	High	Add /auth/forgot-password + /auth/reset-password
Series Episode Tracking	episodes_watched is just a number — no per-episode data	Medium	Add tracked_episodes junction table for granular tracking
API Error Handling	No fallback if TMDB/RAWG APIs are down or rate-limited	High	Add response caching layer + graceful degradation UI
Loading & Skeleton UI	No loading states defined in spec	Medium	Define skeleton components for every data-fetch page
User Onboarding	No first-run experience after registration	Low	Add onboarding modal/tour on first login
SEO / Meta Tags	No mention of Open Graph tags for sharing	Low	Add react-helmet for dynamic meta tags on detail pages
Accessibility (a11y)	No accessibility requirements stated	Medium	ARIA labels, keyboard nav, focus management — defined in Section 9
Activity Feed	Dashboard has "activity history" but schema has no activity table	High	Add activity_log table (see Section 6)

2.2 Architectural Risks

- RAWG API rate limits (20,000 req/month on free tier) — requires request caching
- TMDB API key exposed in backend .env is fine, but must never be sent to frontend
- Render free tier spins down after 15 min inactivity — first request will be slow (cold start). Use a health-check ping or document this behavior for users
- Neon DB free tier has a 0.5 GB storage cap — monitor usage as media IDs accumulate
- No CDN for static assets on Netlify free tier — large poster images from TMDB/RAWG should be proxied or referenced directly from their CDN, not re-hosted

2.3 Decisions Clarified

> *Media metadata (title, poster, description) is NEVER stored in the database. Only IDs (tmdb_id, rawg_id) and user-generated data (status, rating, review) are persisted. All display metadata is fetched live from TMDB/RAWG. This keeps the database lean and always up-to-date.*

> *Neon DB (PostgreSQL serverless) is the chosen database host. The package used in backend is "pg" (node-postgres). This is confirmed and consistent with the architecture.*

3. Functional Requirements

3.1 Authentication & Account Management

ID	Requirement	Priority
FR-01	Users can register with username, email, and password	Must Have
FR-02	Passwords are hashed with bcrypt (salt rounds: 12)	Must Have
FR-03	Login returns a signed JWT access token (24h expiry)	Must Have
FR-04	A refresh token (7d expiry) is issued and stored in DB	Must Have
FR-05	Protected routes reject requests without valid JWT	Must Have
FR-06	Users can change their password (requires old password)	Must Have
FR-07	Users can request a password reset via email	Should Have
FR-08	Users can upload a custom avatar (jpg, png, webp, max 2MB)	Should Have
FR-09	Users can delete their account and all associated data	Should Have

3.2 Media Discovery

ID	Requirement	Priority
FR-10	Users can search movies, series, and games by keyword	Must Have
FR-11	Search results are debounced (300ms) on the frontend	Must Have
FR-12	Trending, popular, and upcoming content loads on the Discover page	Must Have
FR-13	Each media type has a detail page showing full metadata	Must Have
FR-14	Movie detail pages show cast, genres, runtime, and related films	Must Have
FR-15	Game detail pages show platforms, genres, screenshots, and publisher	Must Have
FR-16	TV series pages show seasons, episode count, and airing status	Must Have
FR-17	Search results are paginated (20 items per page)	Must Have

3.3 Library & Tracking

ID	Requirement	Priority
FR-18	Authenticated users can add any media item to their library	Must Have
FR-19	Users can update the status of any tracked item	Must Have
FR-20	Users can remove items from their library	Must Have
FR-21	Users can mark any media item as a favorite	Must Have
FR-22	Users can rate media on a scale of 1–10	Must Have
FR-23	Users can write, edit, and delete text reviews	Must Have
FR-24	Movie/series library can be filtered by status	Must Have
FR-25	Games track hours_played (user-input number field)	Must Have
FR-26	Series track episodes_watched (user-input number field)	Must Have
FR-27	Library is paginated or uses infinite scroll (50 items per load)	Should Have

3.4 Dashboard & Analytics

ID	Requirement	Priority
FR-28	Dashboard shows total counts: movies, series, games per status	Must Have
FR-29	Dashboard shows average rating across all reviewed media	Must Have
FR-30	Dashboard shows a breakdown of favorite genres (top 5)	Should Have
FR-31	Dashboard shows a recent activity feed (last 10 actions)	Should Have
FR-32	Dashboard shows total hours played across all games	Should Have
FR-33	Progress bars show completion % for each media category	Should Have

3.5 Custom Lists

ID	Requirement	Priority
FR-34	Users can create named custom lists	Must Have
FR-35	Users can add any media item (movie, series, game) to a list	Must Have
FR-36	Users can remove items from a list	Must Have
FR-37	Users can delete entire custom lists	Must Have
FR-38	Lists display all added items with their metadata from API	Must Have

4. Non-Functional Requirements

4.1 Performance

- API response time: < 500ms for database queries under normal load
- External API calls (TMDB/RAWG): < 2s, with loading skeleton shown immediately
- Frontend initial load: < 3s on a 4G connection (measured via Lighthouse)
- Search debounce: 300ms to prevent excessive API calls
- Images: Always use TMDB/RAWG CDN URLs directly — never re-host media posters

4.2 Scalability

- Database queries use parameterized statements and indexed foreign keys
- All list endpoints support pagination from day one
- Backend is stateless — JWT auth means horizontal scaling is possible without sessions
- External API responses for trending content are cached for 10 minutes using Node.js in-memory cache (node-cache)

4.3 Security (Summary — Detail in Section 10)

- All passwords hashed with bcrypt, salt rounds: 12
- JWT tokens expire in 24h; refresh tokens in 7 days
- All inputs validated server-side with express-validator
- SQL injection prevented via parameterized pg queries ($1, $2 syntax)
- CORS restricted to production frontend URL only
- Rate limiting on /auth/login: 5 attempts per 15 minutes per IP

4.4 Accessibility

- All interactive elements have visible focus rings
- All images have descriptive alt text
- Color contrast meets WCAG AA (4.5:1 for normal text)
- All form fields have associated <label> elements
- Modal dialogs trap focus and restore it on close
- Keyboard navigation works across all pages

4.5 Browser & Device Support

Target	Minimum Support
Desktop Chrome	Last 2 versions
Desktop Firefox	Last 2 versions
Desktop Safari	Last 2 versions
Mobile Chrome (Android)	Last 2 versions
Mobile Safari (iOS)	iOS 15+
Screen width	320px minimum

5. Software Architecture Design

5.1 System Architecture Overview

Neon Hub follows a classic Client–Server–Database three-tier architecture with external API integration at the service layer.

> *Architecture pattern: REST API backend with stateless JWT authentication. Frontend is a single-page application (SPA) that communicates exclusively via the REST API. External API calls are made exclusively from the backend — never directly from the frontend.*

5.2 Tier Breakdown

Tier	Technology	Host	Responsibility
Presentation	React + Vite + Bootstrap 5	Netlify	UI rendering, routing, state management, API calls to backend only
Application	Node.js + Express.js	Render	Business logic, auth, validation, external API proxy, DB queries
Data	PostgreSQL (via pg)	Neon DB	User data, library entries, reviews, lists, activity logs
External APIs	TMDB API + RAWG API	Third-party	Media metadata: titles, posters, descriptions, cast, genres
File Storage	Cloudinary (free tier)	Cloudinary CDN	Avatar upload, storage, CDN delivery, and auto-resizing via transformation URL params. Persistent across all Render deploys.

5.3 Data Flow — Search & Add to Library

The following describes the complete data flow for the most critical user action: searching for a game and adding it to the library.

1. User types in the search bar on the frontend (React)
2. After 300ms debounce, frontend sends GET /api/games/search?q=batman to the Express backend
3. Express route handler calls RAWG API with the query and the server's API key (never exposed to client)
4. RAWG responds with an array of game objects including rawg_id, title, poster, platforms, genres
5. Backend strips unnecessary fields and returns a clean response array to the frontend
6. User clicks "Add to Library" on a result card
7. Frontend sends POST /api/library/add with { rawg_id, media_type: "game", status: "backlog" } in the request body, plus the JWT in the Authorization header

8. Express middleware verifies the JWT and extracts user_id
9. Controller inserts a row into tracked_games: { user_id, rawg_id, status } — no metadata stored
10. Success response (201) triggers a toast notification and updates the UI button state

5.4 Authentication Flow

11. User submits login form → POST /api/auth/login
12. Server validates credentials, compares bcrypt hash
13. Server issues: access token (JWT, 24h) + refresh token (opaque, 7d, stored in DB)
14. Frontend stores access token in memory (React Context), refresh token in httpOnly cookie
15. On each protected request, access token is sent as Authorization: Bearer <token>
16. When access token expires, frontend calls POST /api/auth/refresh with the refresh token cookie
17. Server validates refresh token against DB, issues new access token
18. On logout, refresh token is deleted from DB, cookie is cleared

5.5 Caching Strategy

Data Type	Cache Layer	TTL	Reason
Trending movies/series	Node in-memory (node-cache)	10 minutes	Rarely changes; saves TMDB quota
Trending games	Node in-memory (node-cache)	10 minutes	Saves RAWG quota
Individual media details	No cache (fetch on demand)	N/A	Low traffic; always fresh
User library data	React state (AuthContext)	Session	Invalidated on any library mutation
Search results	No cache	N/A	Too dynamic to cache effectively

5.6 Project Directory Structure (Final)

Frontend (/client)

src/components/common/ — Reusable atoms: Button, Badge, Loader, Toast, Modal, Pagination, EmptyState

src/components/media/ — MediaCard, GameCard, ReviewCard, StatusDropdown, FavoriteButton, SkeletonCard

src/components/dashboard/ — StatsCard, ProgressBar, ActivityFeed, GenreChart

src/components/layout/ — Navbar, Sidebar, Footer, PageHeader

src/pages/ — One file per page (Home, Discover, MovieDetail, Login, Dashboard, Library, etc.)

src/context/ — AuthContext, LibraryContext, SearchContext, ThemeContext

src/hooks/ — useDebounce, useLocalStorage, useMediaQuery, useLibrary, useAuth

src/services/ — api.js (Axios instance), authService.js, mediaService.js, libraryService.js

src/routes/ — AppRouter.jsx with ProtectedRoute wrapper

src/utils/ — formatDate, truncateText, getStatusColor, calculateCompletionPercent

Backend (/server)

config/ — db.js (pg Pool), cache.js (node-cache instance)

controllers/ — authController, moviesController, seriesController, gamesController, libraryController, reviewsController, listsController, dashboardController

middleware/ — authMiddleware.js (JWT verify), errorHandler.js, rateLimiter.js, cloudinaryMiddleware.js (multer + Cloudinary upload)

models/ — userModel, libraryModel, reviewModel, listModel, activityModel

routes/ — auth.js, movies.js, series.js, games.js, library.js, reviews.js, lists.js, dashboard.js

services/ — tmdbService.js, rawgService.js, emailService.js

validators/ — authValidator, libraryValidator, reviewValidator

utils/ — asyncHandler.js, AppError.js

database/ — schema.sql, seed.sql, migrations/

6. Database Design (Extended)

6.1 Complete Schema

The schema extends the original design with 4 additional tables required for the full feature set.

6.2 Table Definitions

users

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	Auto-increment
username	VARCHAR(50)	UNIQUE NOT NULL	Lowercase, 3–50 chars
email	VARCHAR(255)	UNIQUE NOT NULL	Lowercase, validated format
password	TEXT	NOT NULL	bcrypt hash only
avatar_url	TEXT	DEFAULT '/images/default-avatar.png'	File path or default
email_verified	BOOLEAN	DEFAULT FALSE	For future email verification
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timezone-aware

refresh_tokens (New)

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE	FK to users
token_hash	TEXT	UNIQUE NOT NULL	bcrypt hash of token
expires_at	TIMESTAMPTZ	NOT NULL	7 days from creation
created_at	TIMESTAMPTZ	DEFAULT NOW()	

tracked_movies

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE	
tmdb_id	INTEGER	NOT NULL	TMDB movie ID

status	VARCHAR(20)	CHECK (status IN ('watching','completed','planned','paused','dropped'))	
rating	SMALLINT	CHECK (rating BETWEEN 1 AND 10)	Nullable
review	TEXT	NULL	Optional text review
favorite	BOOLEAN	DEFAULT FALSE	
watched_at	TIMESTAMPTZ	NULL	Set when status → completed
created_at	TIMESTAMPTZ	DEFAULT NOW()	

tracked_series

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE	
tmdb_id	INTEGER	NOT NULL	TMDB series ID
status	VARCHAR(20)	Same CHECK as movies	
rating	SMALLINT	CHECK (rating BETWEEN 1 AND 10)	Nullable
review	TEXT	NULL	
favorite	BOOLEAN	DEFAULT FALSE	
episodes_watched	INTEGER	DEFAULT 0	User-maintained count
created_at	TIMESTAMPTZ	DEFAULT NOW()	

tracked_games

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE	
rawg_id	INTEGER	NOT NULL	RAWG game ID
status	VARCHAR(20)	CHECK (status IN ('playing','completed','backlog','paused','dropped'))	
rating	SMALLINT	CHECK (rating BETWEEN 1 AND 10)	Nullable
review	TEXT	NULL	
favorite	BOOLEAN	DEFAULT FALSE	

hours_played	NUMERIC(6,1)	DEFAULT 0	Allows 0.5-hour precision
created_at	TIMESTAMPTZ	DEFAULT NOW()	

custom_lists

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE	
name	VARCHAR(100)	NOT NULL	e.g. "My Top 10 RPGs"
description	TEXT	NULL	Optional list description
created_at	TIMESTAMPTZ	DEFAULT NOW()	

list_items

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
list_id	INTEGER	REFERENCES custom_lists(id) ON DELETE CASCADE	
media_type	VARCHAR(10)	CHECK (media_type IN ('movie','series','game'))	
media_id	INTEGER	NOT NULL	tmdb_id or rawg_id
added_at	TIMESTAMPTZ	DEFAULT NOW()	

activity_log (New — Required for Dashboard)

Column	Type	Constraints	Notes
id	SERIAL	PRIMARY KEY	
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE	
action	VARCHAR(50)	NOT NULL	e.g. "added_movie", "completed_game", "rated_series"
media_type	VARCHAR(10)	NOT NULL	"movie", "series", or "game"
media_id	INTEGER	NOT NULL	tmdb_id or rawg_id
meta	JSONB	NULL	Optional context e.g. { status: "completed", rating: 9 }

created_at	TIMESTAMPTZ	DEFAULT NOW()	

6.3 Indexes

Table	Index	Reason
tracked_movies	CREATE INDEX ON tracked_movies(user_id)	All library queries filter by user_id
tracked_series	CREATE INDEX ON tracked_series(user_id)	Same reason
tracked_games	CREATE INDEX ON tracked_games(user_id)	Same reason
activity_log	CREATE INDEX ON activity_log(user_id, created_at DESC)	Dashboard feed sorted by recency
refresh_tokens	CREATE INDEX ON refresh_tokens(user_id)	Token lookup on refresh
list_items	CREATE INDEX ON list_items(list_id)	Fetching all items in a list

7. API Contract Reference

7.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register new user. Body: { username, email, password }
POST	/api/auth/login	None	Login. Returns access token + sets refresh token cookie
POST	/api/auth/refresh	Cookie	Exchange refresh token for new access token
POST	/api/auth/logout	JWT	Invalidate refresh token, clear cookie
GET	/api/auth/profile	JWT	Get current user profile
PUT	/api/auth/password	JWT	Change password. Body: { oldPassword, newPassword }
POST	/api/auth/forgot-password	None	Send reset email. Body: { email }
POST	/api/auth/reset-password	None	Reset password. Body: { token, newPassword }

7.2 Movies (TMDB proxy)

Method	Endpoint	Auth	Description
GET	/api/movies/trending	None	Trending movies (cached 10min)
GET	/api/movies/popular	None	Popular movies (cached 10min)
GET	/api/movies/upcoming	None	Upcoming releases
GET	/api/movies/search?q=&page=	None	Search movies. Returns paginated results
GET	/api/movies/:tmdb_id	None	Movie detail: metadata + cast + similar

7.3 TV Series (TMDB proxy)

Method	Endpoint	Auth	Description
GET	/api/series/trending	None	Trending series
GET	/api/series/search?q=&page=	None	Search series
GET	/api/series/:tmdb_id	None	Series detail: seasons, episode count, status
GET	/api/series/:tmdb_id/season/:n	None	Season detail with episode list

7.4 Games (RAWG proxy)

Method	Endpoint	Auth	Description
GET	/api/games/trending	None	Trending / highly-rated games (cached 10min)
GET	/api/games/search?q=&page=	None	Search games
GET	/api/games/:rawg_id	None	Game detail: platforms, genres, screenshots, publisher

7.5 Library

Method	Endpoint	Auth	Description
GET	/api/library	JWT	Get full library. Query: ?type=movies|series|games&status=&page=
POST	/api/library/add	JWT	Add item. Body: { media_type, media_id, status }
PUT	/api/library/update	JWT	Update item. Body: { media_type, media_id, status?, rating?, review?, favorite?, hours_played?, episodes_watched? }
DELETE	/api/library/remove	JWT	Remove item. Body: { media_type, media_id }
GET	/api/library/favorites	JWT	Get all favorited items across all types

7.6 Dashboard

Method	Endpoint	Auth	Description
GET	/api/dashboard/stats	JWT	Aggregate counts, avg rating, total hours played, completion %
GET	/api/dashboard/activity	JWT	Last 20 activity log entries
GET	/api/dashboard/genres	JWT	Top 5 genres by tracked item count (requires API lookups)

7.7 Standard Response Formats

Success

{ "success": true, "data": { ... }, "pagination": { "page": 1, "total": 120, "hasMore": true } }

Error

{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "Email is invalid", "field": "email" } }

HTTP Status Codes Used

Code	Meaning	When Used
200	OK	Successful GET or PUT
201	Created	Successful POST (register, add to library)
400	Bad Request	Validation errors, missing fields
401	Unauthorized	Missing or invalid JWT
403	Forbidden	Valid JWT but wrong user (e.g. editing another user's review)
404	Not Found	Resource does not exist
409	Conflict	Duplicate entry (e.g. registering existing email)
429	Too Many Requests	Rate limit hit
500	Server Error	Unhandled exception — always logged

8. Frontend Architecture & Component Map

8.1 Routing Structure

Route	Page Component	Auth Required	Description
/	Home	No	Landing page: hero + trending content preview
/discover	Discover	No	Browse trending/popular/upcoming by type
/movies/:id	MovieDetail	No	Full movie info, cast, user review section
/series/:id	SeriesDetail	No	Series info, seasons, episode count
/games/:id	GameDetail	No	Game info, platforms, screenshots
/search	SearchResults	No	Search results across all types
/login	Login	No (redirect if auth)	Login form
/register	Register	No (redirect if auth)	Registration form
/dashboard	Dashboard	Yes	Analytics + stats + activity feed
/library	Library	Yes	Tabbed: Movies | Series | Games
/favorites	Favorites	Yes	All favorited items
/lists	CustomLists	Yes	All custom lists
/lists/:id	ListDetail	Yes	Items in a specific list
/reviews	MyReviews	Yes	All user-written reviews
/profile	Profile	Yes	Public-facing user profile
/settings	Settings	Yes	Account settings, avatar upload, password change
*	NotFound	No	404 page

8.2 Context & State Architecture

Context	State Managed	Key Methods
AuthContext	user, accessToken, isLoading	login(), logout(), refreshToken(), updateProfile()
LibraryContext	movies[], series[], games[], isLoading	addToLibrary(), removeFromLibrary(), updateItem(), toggleFavorite()
SearchContext	query, results, isSearching, type	setQuery(), setType(), clearResults()

ThemeContext	theme (dark/light)	toggleTheme()

8.3 Custom Hooks

Hook	Purpose	Returns
useDebounce(value, delay)	Debounce search input by 300ms	debouncedValue
useAuth()	Consume AuthContext safely	{ user, login, logout, isAuthenticated }
useLibrary()	Consume LibraryContext	{ library, addToLibrary, ... }
useMediaQuery(query)	Responsive breakpoints	boolean (matches)
useIntersectionObserver()	Infinite scroll trigger	{ ref, inView }
usePagination(fetchFn)	Generic paginated fetch	{ data, page, hasMore, loadMore }

8.4 Axios Configuration

A single Axios instance (src/services/api.js) handles all HTTP calls with the following setup:

- Base URL: import.meta.env.VITE_API_URL (set to backend URL in .env)
- Request interceptor: Automatically attaches Authorization: Bearer <accessToken> from AuthContext
- Response interceptor: On 401 response, attempts token refresh via POST /auth/refresh, then retries original request once. If refresh fails, calls logout()
- Error interceptor: Normalizes error format to { code, message, field } for consistent UI handling

9. UI/UX Design Specification

9.1 Design Identity

Neon Hub uses a dark, cinematic aesthetic inspired by the glow of screens in a dark room — the physical environment of most serious media consumers. The design language is intentional: deep dark backgrounds, selective neon accents, and clean typographic hierarchy. It should feel premium and focused, not busy.

9.2 Color System

Role	Name	Hex Value	Usage
Primary Accent	Neon Purple	#7B2FBE	CTAs, active states, headings, borders on focus
Secondary Accent	Electric Blue	#00B4D8	Links, hover states, secondary buttons, info badges
Warm Accent	Amber	#F4A261	Ratings, warnings, highlights, favorited state
Background Deep	Void Black	#0D0D1A	Page background, Navbar, Sidebar
Background Surface	Midnight Blue	#1A1A2E	Cards, Modals, Dropdowns
Background Elevated	Deep Navy	#16213E	Hover states on cards, input backgrounds
Text Primary	Ghost White	#E8E8F0	Body text, labels
Text Secondary	Muted Lavender	#9A9AB0	Subtitles, metadata, placeholders
Success	Neon Green	#2D9E5F	Completed status, success toasts
Danger	Crimson	#E53E3E	Error states, dropped status, destructive actions

9.3 Typography

Role	Font Family	Weight	Size	Usage
Display	Space Grotesk	700	2.5rem–4rem	Page titles, hero headlines
Heading	Space Grotesk	600	1.25rem–2rem	Section headers, card titles
Body	Inter	400	1rem	All body text, descriptions
Body Emphasis	Inter	600	1rem	Labels, bold inline text
Caption / Meta	Inter	400	0.8125rem	Release dates, genres, metadata

Data / Stats	Space Grotesk	700	1.75rem–3rem	Dashboard numbers, ratings
Code / IDs	JetBrains Mono	400	0.875rem	API IDs, technical labels

> *All three fonts are available on Google Fonts and load for free. Import via @import in the main CSS file.*

9.4 Component Design Specifications

MediaCard

- Size: 180px wide × 270px tall (poster aspect ratio 2:3)
- Background: Surface (#1A1A2E), border-radius: 12px
- Hover state: Subtle scale(1.03) transform + purple bottom border + overlay with quick-add button
- Shows: poster image, title (truncated at 2 lines), year, rating badge (amber)
- Favorite indicator: amber heart icon, top-right corner, toggles on click
- Status badge: colored pill bottom-left (green = completed, blue = watching, gray = planned)

StatusDropdown

- Renders as a styled Bootstrap dropdown
- Movies/Series statuses with color coding:

Watching	Completed	Planned	Paused	Dropped

- Games statuses:

Playing	Completed	Backlog	Paused	Dropped

StatsCard

- Large number (Space Grotesk 700, 3rem) in neon purple
- Icon (Bootstrap Icon or Lucide) above the number, colored accent
- Label below in muted text (e.g. "Movies Completed")
- Subtle gradient border: linear-gradient(135deg, neonPurple, electricBlue)

Navbar

- Position: fixed top, full width, z-index: 1000
- Background: #0D0D1A with backdrop-filter: blur(10px) for glassmorphism effect
- Logo: "NEON HUB" in Space Grotesk bold with neon purple glow (text-shadow)
- Search bar: centered, expands on focus, collapses on blur (mobile)
- Right: Notifications icon, Avatar dropdown (profile, settings, logout)

Toast Notifications

- Position: bottom-right, stacked vertically
- Types: success (green), error (red), info (blue), warning (amber)

- Auto-dismiss after 4 seconds with progress bar animation
- Manual dismiss via × button

9.5 Page Layouts

Home Page

- Hero: Full-width featured banner (random trending item) with title, tagline, two CTA buttons
- Section: "Trending Movies" — horizontal scroll card row
- Section: "Popular TV Series" — horizontal scroll card row
- Section: "Top Rated Games" — horizontal scroll card row
- Each section has a "See All →" link to the relevant Discover tab

Discover Page

- Tabbed: Movies | TV Series | Games
- Filter bar: Genre dropdown, Sort (Trending / Rating / Release Date), Year range
- Grid: 5 cards per row on desktop, 3 on tablet, 2 on mobile
- Infinite scroll: loads next page when user reaches 80% scroll depth

Detail Page (Movie/Series/Game)

- Hero section: backdrop image (blurred), poster overlay, title, metadata
- Action bar: "Add to Library" button + StatusDropdown + FavoriteButton + Rating stars
- About section: Description, genres, release date, runtime/episodes/platforms
- Cast section (movies/series): horizontal scroll of person cards
- Reviews section: User review form + list of existing reviews
- Similar section: 6 recommendation cards

Dashboard

- Top row: 4 StatsCards — total tracked, avg rating, hours played, completion %
- Middle: Genre breakdown bar chart + Progress rings per media type
- Bottom: Recent activity feed (timeline style, icons by action type)

Library Page

- Three tabs: Movies | Series | Games
- Filter bar: Status filter pills, Sort dropdown
- Grid/List toggle: users can switch between poster grid and compact list view
- Each item: click to open a side drawer with full edit controls (status, rating, review, notes)

9.6 Responsive Breakpoints

Breakpoint	Width	Layout Change
xs (mobile)	< 576px	Single column, hidden sidebar, hamburger menu, 2-card grid

sm (tablet)	576px–767px	2-column grid, condensed navbar
md (tablet landscape)	768px–991px	3-column grid, optional sidebar
lg (desktop)	992px–1199px	4-column grid, full sidebar
xl (wide desktop)	≥ 1200px	5-column grid, full layout

9.7 Empty States

Every data-dependent view must have a defined empty state. Empty states are not errors — they are invitations to act.

View	Empty State Message	CTA
Library (no items)	"Your library is empty. Start adding movies, series, and games."	"Discover Media" button
Favorites	"No favorites yet. Hit the heart icon on any media to save it here."	"Browse Trending" button
Custom Lists	"You haven't created any lists yet."	"Create a List" button
Search (no results)	"No results for '[query]'. Try a different keyword."	"Clear Search" button
Reviews	"You haven't reviewed anything yet. Rate some media to get started."	None

10. Security Implementation Plan

10.1 Authentication Security

Measure	Implementation	Package
Password hashing	bcrypt with 12 salt rounds on register and password change	bcrypt
JWT signing	HS256 algorithm, 24h expiry, signed with JWT_SECRET from .env	jsonwebtoken
Refresh tokens	Opaque token, hashed before storage, 7d expiry, invalidated on logout	crypto + bcrypt
Login rate limiting	5 requests per 15 minutes per IP on POST /api/auth/login	express-rate-limit
Password policy	Min 8 chars, 1 uppercase, 1 lowercase, 1 number — enforced server-side	express-validator

10.2 Input Validation

Endpoint Group	Validated Fields	Rules
Auth register	username, email, password	Username: 3–50 alphanumeric; Email: valid format; Password: policy above
Auth login	email, password	Both present, non-empty strings
Library add/update	media_type, media_id, status, rating	media_type in enum; media_id is integer; rating 1–10 if present
Review create	review text, rating	Text: max 2000 chars; rating: integer 1–10
Avatar upload	File type, file size	MIME: image/jpeg, image/png, image/webp; Size: max 2MB — enforced by multer before Cloudinary upload
Custom list name	name	String: 1–100 chars, non-empty

10.3 Infrastructure Security

Measure	How	Notes
Helmet.js headers	npm install helmet + app.use(helmet())	CSP, X-Frame-Options, MIME sniffing protection
CORS policy	origin: process.env.FRONTEND_URL only	Rejects requests from unknown origins

SQL injection	Parameterized queries only: pool.query("... $1", [val])	Never string interpolation in SQL
Sensitive env vars	Stored in .env, listed in .gitignore, loaded via dotenv	JWT_SECRET, API keys, DATABASE_URL
Cloudinary asset naming	Upload to folder avatars/ with public_id: user-{id}. Cloudinary overwrites on re-upload — no orphaned files	Cloudinary handles naming, CDN delivery, and transformation — no local disk writes
Error messages	Generic messages in production (no stack traces)	Detailed errors logged server-side only

10.4 Environment Variables Checklist

Variable	Where	Example Value
PORT	Backend .env	5000
DATABASE_URL	Backend .env	postgresql://user:pass@host/neonhub
JWT_SECRET	Backend .env	64-char random hex string
TMDB_API_KEY	Backend .env	Obtained from themoviedb.org/settings/api
RAWG_API_KEY	Backend .env	Obtained from rawg.io/apidocs
FRONTEND_URL	Backend .env	https://neonhub.netlify.app
VITE_API_URL	Frontend .env	https://neonhub-api.onrender.com
CLOUDINARY_CLOUD_NAME	Backend .env	Obtained from Cloudinary Dashboard → Account Details
CLOUDINARY_API_KEY	Backend .env	Obtained from Cloudinary Dashboard → API Keys
CLOUDINARY_API_SECRET	Backend .env	Obtained from Cloudinary Dashboard → API Keys. NEVER expose to frontend.
GMAIL_USER=	Backend .env	Gmail account to handle password reset notification
GMAIL_PASS=	Backend .env	Gmail app password

11. Development Phases & Milestones

The 9 phases below expand on the original plan with specific deliverables, acceptance criteria, and estimated effort for a solo developer working with AI agent assistance.

Phase	Name	Est. Duration	Key Deliverables	Done When...
1	Project Setup	1–2 days	/client + /server scaffolded, GitHub repo, Neon DB connected, CORS configured, health check endpoint live	GET /health returns 200 from deployed Render URL
2	Authentication	3–4 days	Register, login, logout, JWT middleware, refresh tokens, password change, protected routes on frontend	All auth flows work end-to-end including token refresh without re-login
3	TMDB Integration	3–4 days	Movies + Series endpoints, TMDB service layer, caching, search with pagination, detail pages	Movie detail page loads cast + metadata; search returns paginated results
4	RAWG Integration	2–3 days	Games endpoints, RAWG service layer, game detail pages, trending games	Game detail page shows platforms + screenshots; search works
5	Library System	4–5 days	Add/update/remove for all types, status system, favorites, activity_log writes, all 3 library tabs	User can add a movie, change its status, rate it, and see it in the dashboard feed
6	Dashboard	3–4 days	Stats endpoint, activity feed, genre analysis, all StatsCards, progress rings, chart	Dashboard shows accurate counts and last 10 activities after library actions
7	Reviews & Lists	3–4 days	Review CRUD, custom lists CRUD, list item management, reviews page, lists detail page	User can create a list, add a movie and game to it, and write a review
8	UI Polish	4–5 days	Skeleton loaders, empty states, toast system, responsive layout, animations, accessibility pass, error boundaries	Lighthouse mobile score > 80; no broken states on any page
9	Deployment	2–3 days	Netlify deploy, Render deploy, Neon DB production, Cloudinary account configured, all environment variables set, smoke test all flows	Full user flow (register → search → add → review → dashboard) works on production URLs

11.1 Pre-Development Checklist

- TMDB API key obtained and tested with a curl request
- RAWG API key obtained and tested with a curl request
- Neon DB project created, connection string saved

- GitHub repository initialized with /client and /server directories
- schema.sql finalized and run on Neon DB — all tables created
- .env files created for both /client and /server, added to .gitignore
- Node.js 20+ installed locally

12. Error Handling & Edge Cases

12.1 Backend Error Handling

All route handlers are wrapped with asyncHandler(fn) utility which catches unhandled promise rejections and passes them to the global error handler middleware. No try-catch blocks scattered across controllers.

Global error handler (errorHandler.js) formats all errors into the standard response format { success: false, error: { code, message } } and suppresses stack traces in production.

12.2 External API Failure Scenarios

Scenario	Frontend Behavior	Backend Behavior
TMDB API down	Show "Content unavailable" banner on affected sections	Return 503 with error code EXT_API_UNAVAILABLE
RAWG rate limit hit	Show "Try again in a moment" message	Return 429 and log the rate limit event
TMDB returns empty results	Show empty state component (not an error)	Return empty array with 200 status
Network timeout	Show retry button after 5s	Axios timeout set to 8s on external API calls
Invalid tmdb_id/rawg_id	Redirect to 404 page	Return 404 with NOT_FOUND code

12.3 Form Validation UX

- Inline field-level errors appear on blur (not on every keystroke)
- Submit button is disabled while a request is in flight (prevents double submission)
- Server-side validation errors map to specific fields (using the "field" key in error response)
- Network errors show a generic "Something went wrong, please try again" toast

12.4 Edge Cases to Handle

Edge Case	Handling
User adds same item twice	Backend returns 409 Conflict; frontend shows "Already in your library" toast
User rates without reviewing	Rating is allowed standalone; review is optional
User deletes account with active library	ON DELETE CASCADE in all FK relationships handles cleanup automatically

JWT expires mid-session	Axios interceptor silently refreshes token; user never sees a logout
TMDB changes poster URL format	Backend constructs image URLs: https://image.tmdb.org/t/p/w500/{poster_path}
Game has no cover image	Frontend fallback: display a styled placeholder with game title
Search query < 2 characters	Frontend does not fire request; shows "Type to search" placeholder
Library is empty on dashboard	Show onboarding prompt: "Add some media to see your stats"
Neon DB cold start (serverless)	pg pool handles reconnection automatically; no special handling needed

13. Testing Strategy

Given the solo/small-team context, the testing strategy is pragmatic: focus on the most critical paths first, automate the high-value tests, and use manual testing for UI/UX flows.

13.1 Backend Testing

Test Type	Tool	Coverage Target	What to Test
Unit Tests	Jest	Core utilities	asyncHandler, AppError, validators, token generation
Integration Tests	Jest + Supertest	All API routes	Auth flows, library CRUD, dashboard stats calculation
Manual API Testing	Postman/Thunder Client	All endpoints	Happy path + error cases for every route

13.2 Frontend Testing

Test Type	Tool	Coverage Target	What to Test
Component Tests	Vitest + React Testing Library	Common components	MediaCard render, StatusDropdown options, AuthContext state
E2E Tests (Phase 8+)	Playwright	Critical user flows	Register → login → add to library → view dashboard
Manual QA	Browser + DevTools	All pages	Responsive layout, empty states, loading states

13.3 Pre-Deployment Checklist

- All API endpoints tested in Postman with valid and invalid inputs
- Auth flow: register, login, protected route, refresh, logout — all work
- Library: add, update status, rate, favorite, remove — all media types
- Dashboard stats update correctly after library changes
- Custom lists: create, add items, remove items, delete list
- Search: results appear, debounce works, pagination loads next page
- Detail pages: metadata loads, "Add to Library" works from detail page
- Mobile layout: all pages render correctly at 375px width
- Accessibility: all interactive elements focusable via keyboard, no missing alt text
- Performance: Lighthouse score > 80 on mobile for Home and Discover pages

14. Deployment & DevOps

14.1 Deployment Stack Summary

Service	Provider	Free Tier Limits	Notes
Frontend	Netlify	100GB bandwidth/month, unlimited deploys	Set VITE_API_URL env var in Netlify dashboard. Add netlify.toml with SPA redirect rule.
Backend	Render	750h/month, spins down after 15min inactivity	Set all .env vars in Render environment settings
Database	Neon DB	0.5GB storage, 1 project, 10 branches	Use the connection string as DATABASE_URL
File Storage (Avatars)	Cloudinary	25 GB storage, 25 GB bandwidth/month, 25,000 transformations/month — permanently free	Avatars uploaded via multer-storage-cloudinary. secure_url stored in DB. Auto-resized to 200×200px via transformation URL params. No files on Render's ephemeral filesystem.
File Storage (avatars)	Cloudinary	25 GB storage, 25 GB bandwidth/month, 25,000 transformations/month — permanently free	Avatars uploaded via multer-storage-cloudinary. Secure URLs stored in avatar_url column. Auto-resized to 200×200px via Cloudinary transformation URL parameters — no Sharp needed

> *Cloudinary is the confirmed file storage provider for all user avatar uploads. The free tier provides 25 GB storage and 25 GB bandwidth per month — more than sufficient for a production app at this scale. No files are stored on Render's ephemeral filesystem. The backend uses the multer-storage-cloudinary package, which pipes the uploaded file directly from multer's memory buffer to Cloudinary without writing to disk. The returned secure_url is stored in the avatar_url column of the users table. Cloudinary's transformation URLs resize the avatar to 200×200px on delivery — no Sharp or image processing library is needed on the server.*

14.2 Frontend Deployment (Netlify)

19. Push /client to GitHub (can be in a monorepo or separate repo)
20. Connect repo to Netlify, set Base Directory: client, Build Command: npm run build, Publish Directory: dist
21. In Netlify > Site Settings > Environment Variables, add: VITE_API_URL = https://your-render-app.onrender.com
22. Add a netlify.toml file in /client with redirect rule: [[redirects]] from = "/*" to = "/index.html" status = 200 (required for React Router to work on refresh/direct URL)

14.3 Backend Deployment (Render)

23. Connect /server directory to Render as a "Web Service"
24. Set Start Command: node server.js (or npm start)
25. Add all environment variables from Section 10.4 in Render dashboard
26. Set Health Check path: /health (add a simple GET /health → 200 endpoint to server.js)
27. After deploy, run schema.sql on the Neon DB using the Neon web console SQL editor

14.4 Cloudinary Setup (File Storage)

Cloudinary handles all avatar uploads with persistent, CDN-delivered storage. The free tier (25 GB storage, 25 GB bandwidth/month) is more than sufficient. No credit card is required to sign up.

28. Create a free account at cloudinary.com — no credit card required
29. From the Cloudinary Dashboard, copy: Cloud Name, API Key, API Secret
30. Add CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET to Render environment variables (same dashboard as other env vars)
31. In the backend, install: npm install cloudinary multer-storage-cloudinary multer
32. cloudinaryMiddleware.js configures a CloudinaryStorage instance with folder: “avatars”, allowed formats: [“jpg”, “png”, “webp”], and public_id: (req) => “user-” + req.user.id. Multer uses this storage, limiting file size to 2 MB.
33. After upload, req.file.path contains the Cloudinary secure_url. Store this in users.avatar_url. To auto-resize on delivery, append Cloudinary transformation parameters to the URL: /c_fill,g_face,h_200,w_200/ before the file name segment.

14.5 Keeping Render Alive (Free Tier)

Render free services spin down after 15 minutes of inactivity, causing ~30-second cold starts. Mitigation options:

- Option A: Add a comment in the README documenting this behavior (simplest)
- Option B: Use a free uptime monitor (e.g., UptimeRobot) to ping GET /health every 14 minutes, keeping the service warm
- Option C: Upgrade to Render Starter ($7/month) once the project has real users

15. Future Enhancements (v2+)

These features are intentionally out of scope for v1 but are designed for without blocking their future addition.

15.1 v2 Feature Candidates

Feature	Value	Complexity	Notes
Social Follow System	See what friends are watching/playing	High	Requires followers table + activity feed scoping
Public Profile Pages	Share your library and stats publicly	Medium	Requires visibility settings on tracked items
Notification System	Alert for new episodes, game releases	Medium	Requires notification_prefs + cron job + email service
Advanced Genre Analytics	Radar chart of genre diversity, heatmaps	Medium	Requires storing genre IDs from TMDB/RAWG on tracking
Mobile App (React Native)	iOS + Android native experience	Very High	Shared API backend, same endpoints
AI Recommendations	Personalized suggestions based on library	High	Claude API integration for intelligent matching
Streaming Availability	Show which platforms each movie is on	Low	TMDB watch providers endpoint already available
Import from Letterboxd/MyAnimeList	Migrate existing tracking data	Medium	CSV or API import flow
Moderation & Admin Panel	Flag/remove inappropriate reviews	Medium	Admin role in users table + admin-only routes

15.2 Technical Debt to Address in v2

- Move from in-memory caching (node-cache) to Redis for distributed caching when scaling beyond one Render instance (Redis free tier available via Upstash)
- Replace pg (raw SQL) with Prisma ORM for type-safe database access and easier migrations
- Add a proper migration system (e.g., db-migrate or Flyway) to manage schema changes over time
- Replace Bootstrap 5 with Tailwind CSS for more precise design control and smaller bundle
- Add OpenTelemetry for distributed tracing and proper production observability